



STM32 Firmware

GENERATING 32-BIT STM32 UNIQUE ID

The **STM32 Unique ID** is 96 bits long (12 bytes). For most applications, it is not useful to have such a long unique ID because of a limited number of units in circulation. However, it might be extremely important for you to have a unique ID that does not match with another device in the lot – at least has a VERY LOW chance of matching another device.

The expectations when trying to figure out this hashing method were the following:

- Must be **fast and easy**
- Should take **less than 512 bytes flash and 128 bytes RAM**
- Should be **hardware-independent**
- **Single C and H file** – no mess
- 32-bit unique ID must not repeat in a lot of, say, 10,000 consecutive UID pcs.

Structure of the 96-bit unique ID

The 96-bit unique ID present on the STM32 chips (at least the STM32L4 series) is structured in a specific way. This information is not specified for all STM32 chips, but I would assume that this is how their UIDs are put together as well.

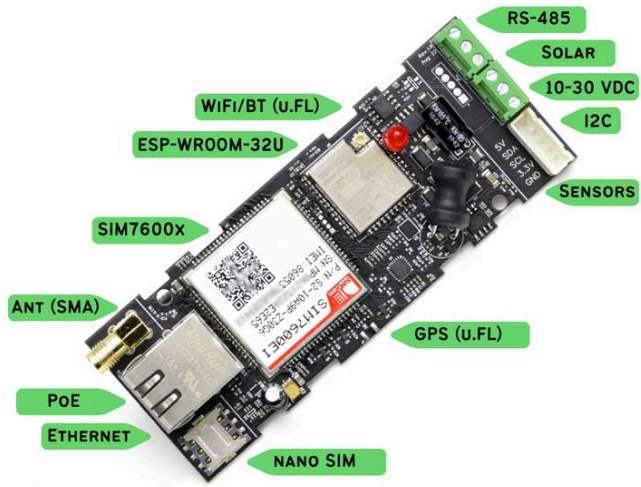
The [STM32L4 reference manual \(RM0351\)](#) breaks down the 96-bit unique ID structure as follows:

- **UID[95:64]:** LOT_NUM[55:24] – Lot number (ASCII encoded)

Type and hit enter...



NEW PRODUCTS



INTRODUCING ESP32 4G LTE GATEWAY

CONNECT EVERYTHING.
POWERED BY ANYTHING.

- **UID[39:32]:** WAF_NUM[7:0] – Wafer number (8-bit unsigned number)
- **UID[63:40]:** LOT_NUM[23:0] – Lot number (ASCII encoded)
- **UID[31:0]:** X and Y coordinates on the wafer

As you can see, none of the above data fields can be extracted and put together as a 32-bit STM32 unique ID without the risk of repitition of the 32-bit UID within a batch of 10,000 devices.

Also, considering the fact that people have noticed that the X-Y coordinates are rather inconsistent and may contain a lot of 0’s in the MSB, especially for chips with large wafer size, which means lot number + coordinate data is not unique at all.

CityHash Code for Generating 32-bit STM32 Unique ID

A hashing function is the perfect solution to this problem. A hashing function maps the input data string to a (usually) shorter data string while trying to distribute the mapping as evenly as possible. With a good hashing function, a single bit changing in the input string will give you a completely different output string. The main characteristic that we are looking for in the hashing function here is that the **repetition in the output string should be evenly distributed and minimized, given a changing longer input string.**

Google’s CityHash is a great solution to the above problem. The string hashing logic is simple, computationally light weight, low footprint and easy to use. CityHash is optimized for hashing strings.

The following C code (H file and C file) is enough to get the job done. You simply need to feed in a string with 12 bytes and the CityHash algorithm produces a unique 32-bit UID from the STM32 UID register contents.

Note that this is not the complete implementation of the CityHash logic. I have removed several conditional compilation clauses and the logic that works on longer data input strings in order to save flash space.

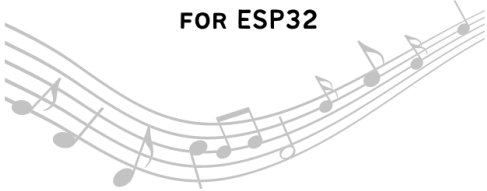
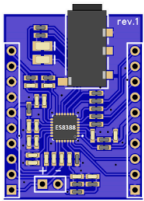
This code will only work on 5-byte to 12-byte long input fields.

Header file (**stm32_uidhash.h**):

ESP-ADF COMPATIBLE

AUDIO CODEC MODULE

FOR ESP32



AVAILABLE TO ORDER!

QUICK CONTACT

Your Name

E-mail

Your question or message

SEND MESSAGE

```
1  #ifndef _STM32_UIDHASH_
2  #define _STM32_UIDHASH_
3
4  uint32_t Hash32Len5to12(const char *s, size_t len);
5
6  #endif
```

C file (**stm32_uidhash.c**):

```
1  #include <stdio.h>
2  #include <string.h>
3
4  #include "stm32_uidhash.h"
5
6  // Magic numbers for 32-bit hashing. Copied from Murmur3.
7  static const uint32_t c1 = 0xcc9e2d51;
8  static const uint32_t c2 = 0x1b873593;
9
10 static uint32_t UNALIGNED_LOAD32(const char *p) {
11     uint32_t result;
12     memcpy(&result, p, sizeof(result));
13     return result;
14 }
15
16 static uint32_t Fetch32(const char *p) {
17     return UNALIGNED_LOAD32(p);
18 }
19
20 static uint32_t Rotate32(uint32_t val, int shift) {
21     // Avoid shifting by 32: doing so yields an undefined result.
22     return shift == 0 ? val : ((val >> shift) | (val << (32 - shift)));
23 }
24
25 // A 32-bit to 32-bit integer hash copied from Murmur3.
26 static uint32_t fmix(uint32_t h)
27 {
```

```

28     h ^= h >> 16;
29     h *= 0x85ebca6b;
30     h ^= h >> 13;
31     h *= 0xc2b2ae35;
32     h ^= h >> 16;
33     return h;
34 }
35
36 static uint32_t Mur(uint32_t a, uint32_t h) {
37     // Helper from Murmur3 for combining two 32-bit values.
38     a *= c1;
39     a = Rotate32(a, 17);
40     a *= c2;
41     h ^= a;
42     h = Rotate32(h, 19);
43     return h * 5 + 0xe6546b64;
44 }
45
46 uint32_t Hash32Len5to12(const char *s, size_t len) {
47     uint32_t a = (uint32_t)len, b = a * 5, c = 9, d = b;
48     a += Fetch32(s);
49     b += Fetch32(s + len - 4);
50     c += Fetch32(s + ((len >> 1) & 4));
51     return fmix(Mur(c, Mur(b, Mur(a, d))));
52 }

```

Using the code is easy. All you need to do is pass an array with STM32 unique ID register values as a 12-character array.

The value returned is the 32-bit hashed UID.

Here is a code snippet showing how to use the CityHash function on an **STM32G4**:

```

1 char uidstr[12];
2
3 // Arrange 12 bytes of UID into uidstr[]
4 uint32_t uid = LL_GetUID_Word0 ();
5 memcpy (&uidstr[8], &uid, 4);

```

```
6 | uid = LL_GetUID_Word1 ();
7 | memcpy (&uidstr[4], &uid, 4);
8 | uid = LL_GetUID_Word2 ();
9 | memcpy (&uidstr[0], &uid, 4);
10|
11| // Generate UID value from uidstr[]
12| uid = Hash32Len5to12 ((const char *)uidstr, 12);
```

The code footprint is tiny, only about 400 bytes even with optimization level set to debug.
For comparison, HAL peripheral init routines are usually larger than the hashing logic.

Memory RegionsMemory Details

Selection:404 B

Search

Name	Run address (VMA)	Load address (LMA)	Size	
MX_DMA_Init	0x08001020	0x08001020	60 B	
MX_GPIO_Init	0x0800105c	0x0800105c	340 B	
HAL_TIM_PeriodElapsedCallback	0x080011b0	0x080011b0	36 B	
Error_Handler	0x080011d4	0x080011d4	14 B	
UNALIGNED_LOAD32	0x080011e2	0x080011e2	28 B	
Fetch32	0x080011fe	0x080011fe	24 B	
Rotate32	0x08001216	0x08001216	40 B	
fmix	0x08001240	0x08001240	80 B	
Mur	0x08001290	0x08001290	92 B	
Hash32Len5to12	0x080012ec	0x080012ec	140 B	
HAL_Msplnit	0x08001378	0x08001378	80 B	
HAL_SAI_Msplnit	0x080013c8	0x080013c8	352 B	
HAL_InitTick	0x08001528	0x08001528	244 B	
NMI_Handler	0x0800161c	0x0800161c	6 B	
HardFault_Handler	0x08001622	0x08001622	6 B	
MemManage_Handler	0x08001628	0x08001628	6 B	

Generating 32-bit STM32 Unique ID, CityHash Flash Usage

Tried and Tested on our Products

We have a bunch of products out there that use this hashing algorithm to reduce the 96 bits down to a short 32-bit STM32 unique ID. The [PCB Artists sound level meter](#) is one such example. The device contains a 32-bit unique ID that it generates from the 96-bit factory programmed UID.

As we determined above, there is **little to no chance of having the same 32-bit STM32 unique ID in a lot of around 10,000 devices**, especially when they are parts from the same manufacturer reel with consecutive serial numbers.



Have Something to Say?

Feel free to ask away via the **Live Chat**, drop us a message using the **Quick Contact** form in the sidebar, or leave a **comment** below.

Change Log

- **24 May 2023**
– Initial release

STM32

1 comment 1 ♥ f X P ✉

YOU MAY ALSO LIKE

SOUND LEVEL SENSOR MODULE FOR
ARDUINO

USING 4 LANE MIPI DSI DISPLAY
◉ ◉ WITH STM32 ◉ ◉

SOLVED: UNDEFINED REFERENCE TO
FUNCTION (STM32 CUBEIDE)

LEAVE A COMMENT

Your Comment

Name*

Email*

Website

☐ Save my name, email, and website in this browser for the next time I comment.

Please enter an answer in digits:

four + three =

SUBMIT

1 COMMENT

EERO AF HEURLIN

REPLY

🕒 January 17, 2025 - 12:07 pm

Many thanks for this code, I made a port to MicroPython it's published here
<https://gist.github.com/rambo/018b55fb82dad66ac8a8188b2b7c59d1>



LINKEDIN



EMAIL



GITHUB

DISCLAIMER: The information provided on this website is for informational use only. PCB Artists provides no guarantees or warranty to the information contained.
@2019-2023 - All Right Reserved. Designed and Developed by PCB Artists (OPC) Private Limited.

[Store Terms and Conditions](#) • [Privacy Policy](#) • [Shipping Policy](#)

^
BACK TO TOP